

AFRILANG Stdlib

std/pathutil

Français. Bibliothèque AFRILANG 'std/pathutil' (niveau simple, catégorie system). Elle expose 2 fonction(s) : normalizeSlashes, hasTrailingSlash.

'import "std/pathutil"' · 'use pathutil'

- 'normalizeSlashes(s text) → text' - 'hasTrailingSlash(s text) → bool' **En-**

glish. AFRILANG library 'std/pathutil' (tier simple, category system). Exports 2 function(s): normalizeSlashes, hasTrailingSlash.

'import "std/pathutil"' · 'use pathutil'

- 'normalizeSlashes(s text) → text' - 'hasTrailingSlash(s text) → bool'

Catégorie / Category	Système	
Niveau / Tier	simple	June 16, 2026
Fonctions / Functions	2	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/pathutil' (niveau simple, catégorie system). Elle expose 2 fonction(s) : normalizeSlashes, hasTrailingSlash.

'import "std/pathutil"' · 'use pathutil'

```
- `normalizeSlashes(s text) → text`
- `hasTrailingSlash(s text) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/pathutil"
use pathutil
```

Niveau : simple · Catégorie : Système
CPU, disque, processus, environnement.

3. Référence API

- normalizeSlashes(s text) → text
hasTrailingSlash(s text) → bool

4. Exemples d'utilisation

```
import "std/pathutil"
use pathutil
```

```
create result = normalizeSlashes(/* args */)
say result
```

```
create result = hasTrailingSlash(/* args */)
say result
```

5. Bonnes pratiques

- Préférez 'import "std/pathutil"' en tête de fichier.
Lancez 'afri-lang check' avant de livrer.
Combinez avec les modules stdlib de la même catégorie.
Voir /stdlib/ pour le source live et les démos playground.
Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module pathutil

```
function normalizeSlashes(s text) returns text
end
```

```
function hasTrailingSlash(s text) returns bool
end
end
```

English

1. Overview

AFRILANG library 'std/pathutil' (tier simple, category system). Exports 2 function(s): normalizeSlashes, hasTrailingSlash.

'import "std/pathutil"' · 'use pathutil'

```
- `normalizeSlashes(s text) → text`
- `hasTrailingSlash(s text) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/pathutil"
use pathutil
```

Tier: simple · Category: System
CPU, disk, processes, environment.

3. API reference

- normalizeSlashes(s text) → text•
hasTrailingSlash(s text) → bool

4. Usage examples

```
import "std/pathutil"
use pathutil
```

```
create result = normalizeSlashes(/* args */)
say result
```

```
create result = hasTrailingSlash(/* args */)
say result
```

5. Best practices

- Prefer `import "std/pathutil"` at the top of your file. •
Run `afrilang check` before shipping. •
Combine with related stdlib modules from the same category. •
See /stdlib/ for the live source and playground demos. •
Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module pathutil

```
function normalizeSlashes(s text) returns text
end
```

```
function hasTrailingSlash(s text) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/pathutil"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/pathutil/>

Source (extrait)

```
module pathutil

function normalizeSlashes(s text) returns text
end

function hasTrailingSlash(s text) returns bool
end
end
```